RNNs for the 2nd argmax *

Zihao Chen †

April 2024

1 Introduction

The goal of this work is to perform mechanistic interpretation analysis on an algorithmic task, aiming to develop a heuristic estimate through this analysis. Ideally, we hope to use mechanistic analysis to extract the high-level algorithm and employ heuristic arguments to address any remaining fuzziness, providing an estimate of the model's accuracy.

In this note, we discuss progress towards this goal for the task of finding the index of the second largest number in a sequence, which we refer to as the "2nd argmax task" throughout. We train vanilla one-hidden-layer recurrent neural networks (RNNs) for this task. Intuitively, the concept of max or argmax should emerge from these networks, which is empirically confirmed. Hence, we begin by training linear RNNs on the argmax task as a step towards the 2nd argmax task.

2 Problem setup

Given a sequence of real numbers $x = (x_1, x_2, ..., x_d)$, where each entry is sampled i.i.d. from the uniform distribution Unif$[-1, 1]$, we train simple RNNs to solve the d-way classification task of predicting the maximum entry (argmax) and the second-largest entry (2nd argmax) in the sequence. We consider the simple class of one-hidden-layer RNNs. More specifically, the RNN architecture is given by the following equations:

$$\begin{aligned} h_0 &= \mathbf{0} \\ h_{t+1} &= \mathrm{ReLU}(W_{hh} h_t + x_t w_{ih}) \\ y &= W_{oh} h_{d+1}, \end{aligned}$$

where $h_t \in \mathbb{R}^m$ is the hidden state at time t, initialized with $h_0 = 0$. $w_{ih} \in \mathbb{R}^m$, $W_{hh} \in \mathbb{R}^{m \times m}$ are the input weights and recurrent weights for the hidden layer, respectively. The ReLU activation function is applied element-wise. The output $y \in \mathbb{R}^d$ represents the logits

*A complimentary notebook can be found at `https://colab.research.google.com/drive/1KgkLyR-sKB8PZxYtjH9YIdgJPus0I8tW?usp=sharing`

†Email: zihaochen1996@gmail.com

for the d-way classification at time t, obtained by a linear transformation with weights $W_{oh} \in \mathbb{R}^{d \times m}$. We do not have any bias terms throughout the network. The RNNs are trained using cross-entropy loss (with respect to the one-hot encoding of the class labels) to predict the correct argmax and 2nd argmax classes.

We would like to make the following observations of the setup and the model.

Generalization of the input range The RNN architecture with ReLU activations and without biases exhibits a generalization property with respect to the input range. Since ReLUs are positive homogeneous functions, i.e., $\text{ReLU}(\alpha x) = \alpha \text{ReLU}(x)$ for any $\alpha > 0$, the network's computations are equivariant to positive scaling of the input sequence. Specifically, if we scale the input sequence $(x_1, x_2, ..., x_d)$ by any positive constant α, the output will scale by α as well. This implies that the network's behavior generalizes beyond the $[-1, 1]$ input range, maintaining the same accuracy across all symmetric uniform distributions.

Equivariance and invariance The RNN exhibits an equivariance property with respect to the positive scaling of the hidden state representations. Specifically, let $D \in \mathbb{R}^{m \times m}$ be a full-rank positive diagonal matrix, and define the scaled hidden state $h'_t = Dh_t$. Then, there exists an equivalent RNN with modified weights. Let $W'_{hh} = DW_{hh}D^{-1}$, $w'_{ih} = Dw_{ih}$, $W'_{oh} = W_{oh}D^{-1}$ and h'_t be the transformed weights. It can be shown that:

$$h'_t \equiv Dh_t$$

and

$$y'(x_1, x_2, ..., x_d) = W'_{oh}h'_{d+1} \equiv W_{oh}h_{d+1}.$$

The model also exhibits invariance with respect to the output matrix W_{oh}. Note that $W'oh = Woh + \mathbf{1}\mathbf{u}^T$ has exactly the same model output, loss, and accuracy as W_{oh} due to the shifting-invariance of the softmax function. With strong weight decay, we should expect each column of W_{oh} to sum to 0, implying the average of all output directions is the $\mathbf{0}$ vector. However, our main RNN model for the 2nd argmax task was not trained with very strong weight decay for generalization concerns.

Accuracy of intermediate logits Since the network does not have bias terms, the model output after receiving the input sequence $(0, 0, ..., 0, x_1, x_2, ..., x_t)$ is identical to receiving $(x_1, x_2, ..., x_t)$ at time t. This implies the intermediate logit values should reflect the running classification result. For example, for the argmax task, we should expect the intermediate logits to reflect the current max index, even though the training cross-entropy loss does not explicitly include the intermediate logits. A similar rationale holds for the 2nd argmax task as well. However, the intermediate accuracies may not be monotonically decreasing since we don't explicitly train for the accuracy when the inputs start with 0s

Spurious correlation and backdoors Note that with high probability, the max and the 2nd max values are positive. Therefore, the neural networks are not incentivized to perform well on purely negative sequences, an effect further enhanced by the ReLU

activation. Consequently, we expect the trained networks to perform well on benign distribution shifts like $\text{Unif}[0,1]^d$ but not on $\text{Unif}[-1,0]^d$.

One can also poison the data by flipping the sign of one particular index on the distribution $\text{Unif}[-1,1]^d$. This results in a $1/d$ accuracy drop on the original distribution, which may be less noticeable when d is large.

3 Linear RNN for Argmax

In the simpler case of a linear RNN, we remove the nonlinearity from the architecture and focus on the scenario where $m \geq d$. The hidden state updates become $h_{t+1} = W_{hh}h_t + x_t w_{ih}$. Without the nonlinearity, the hidden state dynamics and the output logits are linear functions of the input sequence $x = (x_1, x_2, ..., x_d)$. We can write the overall input-output mapping as a linear function $y = Ax$, where $A \in \mathbb{R}^{d\times d}$ is a matrix that encapsulates the linear transformation performed by the RNN:

$$A = W_{oh}\begin{bmatrix} W_{hh}^{d-1}w_{ih} & W_{hh}^{d-2}w_{ih} & \cdots & w_{ih}\end{bmatrix}.$$

The matrix A represents the linear mapping from the input sequence x to the output logits y, which is obtained by multiplying the input weight vector w_{ih} with successive powers of the recurrent weight matrix W_{hh}, and then applying the output weight matrix W_{oh}.

The extracted matrix A is also the covariance between the logits and input (up to scaling). Note that, in order for the argmax of the output logits y to always coincide with the argmax of the input sequence x, the matrix A must satisfy :

$$A = cI + \mathbf{1}\mathbf{u}^T$$

where c is a scalar, I is the $d \times d$ identity matrix, $\mathbf{1}$ is the d-dimensional vector of ones, and u is a d-dimensional vector. This representation ensures that the linear transformation performed by A preserves the ordering of the input elements while allowing for a constant shift and scaling.

The specific values of the RNN weights W_{hh} and w_{ih} are not crucial, as long as the concatenated matrix $[W_{hh}^{d-1}w_{ih}\ W_{hh}^{d-2}w_{ih}\ ...\ w_{ih}]$ has full rank. In fact, we can simply set $W_{oh} = [W_{hh}^{d-1}w_{ih}\ ...\ w_{ih}]^\dagger$ to obtain $A = I$ directly. For instance, we can even take W_{hh} and w_{ih} as random, although the condition number of the concatenated matrix exponentially blows up and the learning dynamics do not favor these solutions. Empirically, W_{hh} tends to be a cyclical shift in the directions specified by the output matrix W_{oh}.

3.1 A naive heuristic estimate

Based on the previous observation regarding the extracted linear transformation A, we can obtain a simple heuristic estimate of the classification accuracy:

1. For a given A extracted from the trained network, we find the best approximation of the form $(D + \mathbf{1}\mathbf{u}^T)$ in the Frobenius norm, where D is a diagonal matrix.

2. We use $\mathbb{P}[\arg\max Dx = \arg\max x]$ as the accuracy estimate. This can be analytically computed for the uniform distribution.

The diagonal matrix D represents how well the linear RNN decodes the values of each direction/index. This procedure doesn't account for the approximation error of the $\mathbf{1u}^T$ (applying the presumption of independence) and thus tends to overestimate the accuracy.

Despite its simplicity, this heuristic estimate can be used for detecting some backdoors. For instance, suppose the linear RNN was trained to classify $\arg\max(x_1, ..., -x_p, ..., x_d)$ for some specific poisoned index p. The resulting RNN can have a decent accuracy up to $1 - 1/d$, but it will have a negative D_{pp}, which can be detected based on the estimate procedure.

4 Nonlinear RNN for Argmax

Since the argmax task can be perfectly solved by a linear RNN with m neurons, the benefits of non-linear networks mainly lie in representation efficiency (i.e., fewer neurons are required). To perfectly solve the task, the network needs to learn a $(d-1)$-dimensional vector representing the differences among the input sequence, which requires at least $d-1$ neurons for linear networks.

Let's consider a few scenarios that may require different explanations:

1. A trained linear RNN with the same number of neurons as the input degree of freedom, i.e., $m = d - 1$.

2. We first obtain a linear RNN by randomly sampling the weights W_{hh} and w_{ih}. Then we take W_{oh} as the inverse of the aforementioned concatenated matrix yielding a perfect linear RNN. Next, we add ReLU to this network, but it can still obtain a decent accuracy. [1]

3. A trained RNN with $m = d - 1$ neurons.

4. A trained RNN with $m \leq d - 2$ neurons.

The first is well understood. For the second, we need to explain why ReLUs are benign distortions. The third and fourth likely require different explanatory mechanisms. Our goal was full mechanistic interpretations for all cases, but this quickly became challenging when nonlinearities played a key role. As an example, if the network simply approximates the identity, mechanistically explaining how may be less important. Instead, treating it as a black box and arguing the input-output mapping could suffice. Ideally, a generic heuristic estimator (e.g., bin propagation [2]) could justify such components. However, we avoided involving these generic procedures and focused on attempts that are potentially parochial. We do not expect covariance propagation to work well in this setting due to several layers of nonlinearity.

[1] For example, if we sample all entries of W_{hh} and w_{ih} i.i.d. from Unif$[0, 1]$, we empirically find this holds for $d = 10$.

[2] For instance, one could take the conditional distribution of the k-th entry being the argmax for each k and then use sampling to obtain the bins and propagate to estimate the k-th softmax value.

4.1 Handcoded Nonlinear RNN

It is straightforward to construct a ReLU RNN with $2d$ neurons that can implement the identity function and thus achieve perfect accuracy. We can set the input weight vector w_{ih} to contain a 1 and -1 with zeros elsewhere so that we can extract the latest input x_t as $\mathrm{ReLU}(x_t) - \mathrm{ReLU}(-x_t)$. The remaining $2d-2$ neurons are used for shifting and storing these values. We can also reduce the total number of neurons to d by only computing and storing $\mathrm{ReLU}(x_t)$, which incurs an exponentially small error rate. Indeed, there are other constructions of computing the argmax without preserving the total order of the sequence. These could serve as a starting point for gaining intuition about the trained networks.

4.2 Attempts of heuristic estimators

We present a few ideas on heuristic estimators. However, all of them are based on extracting linear approximations of the nonlinear network, and thus do not work well in the undercomplete setting ($m \leq d-2$). These estimators fail to explain the advantages provided by nonlinearity, instead, viewing it as a distortion from the linear case.

- One naive approach is to ignore the nonlinearity and treat the network as a linear RNN. If this is a good approximation, it would imply that the nonlinearity is a benign distortion. Empirically, we find this approximation is better when $m \geq d$ and d are small.
- A minor improvement over the previous estimate is to note that $h_1 = \mathrm{ReLU}(x_0 w_{ih})$ is either $x_0 \mathrm{ReLU}(w_{ih})$ or $-x_0 \mathrm{ReLU}(-w_{ih})$, depending on the sign of x_0. So, we could replace w_{ih} by $\frac{1}{2}(\mathrm{ReLU}(w_{ih}) - \mathrm{ReLU}(-w_{ih})) = \frac{1}{2} w_{ih}$ while keeping the higher-order columns unchanged in the extracted concatenated matrix $[W_{hh}^{d-1} w_{ih} \ \cdots \ w_{ih}]^{\dagger}$.
- Another approximation is using $\sum_{i=1}^{d} x_i h_{d+1}(e_i)$ to approximate the final hidden state $h_{d+1}(x)$, where $e_i = (0, \ldots, 1, \ldots, 0)$ is the sequence represented by the i-th standard basis vector, and $h_{d+1}(x)$ represents the final hidden state value with input sequence x. Intuitively, this aims to extract how the network depends on x_i when x_i is large and marginally combine them as an approximation.

5 Nonlinear RNN for 2nd Argmax

For the 2nd argmax task, nonlinearity and more neurons are required. In this section, we focus on our analysis of a 16-neuron RNN that achieves 94% accuracy on the uniform distribution for sequence length $d = 10$. We sketch some findings of the trained network and a hand-coded network inspired by these findings. Visualizations and details can be found in the notebook.

5.1 Mechanistic Findings

We summarize a few more definitive findings of the network, which we will use as hints to construct an almost-perfect RNN.

1. We have several major input neurons: N1, N4, N6, N7, N8. The weight for N4 is positive, while the others are negative.

2. N2 approximately implements the running max value $\max_t$, N4 approximately implements $\text{ReLU}(x_t - \max_t)$, and N7 approximately implements $\text{ReLU}(\max_t - x_t)$.

3. N8 approximately implements $\text{ReLU}(2\text{ndmax}_t - x_t)$ in earlier time steps.

4. When calculating the logits at each time t, projecting out the negative input neurons (N1, N6, N7, N8) gives an argmax/max sub-circuit that accurately finds the max index.

5. At a high level, new values enter through the last logit (m-th), and the previous values (inexactly) shift to earlier logits. For each iteration t, if $x_t > \max_t$ (running max value), the old max logit gets a boost. Note that this restates that the intermediate logits have good accuracies for the running 2nd argmax.

5.2 The handcoded RNN

The high-level idea is to maintain a set of features, m_k and s_k, for representing the scores of each index as the max and the 2nd max index, respectively. They correspond to the intermediate logit values and the projected logit values in the trained network. In other words, $\arg\max m_k$ is the max index, and $\arg\max s_k$ is the 2nd max index at each time step.

When the new input x_t is larger than the running max, i.e., $x_t > \max_t$, we need to update the s value for the current argmax logit. Mimicking the trained network can be achieved by shifting the m values to the s values. We provide two ways of implementing this shift in the code: the exact shifting and the approximate shifting. The exact shifting is conceptually easier to understand, while the approximate shifting uses ReLUs to implement a step function. The approximate shifting version can be exactly implemented by a ReLU RNN.

A simple choice for m_k is x_k. The choice of $s_9(x_t)$ is inspired by the trained RNN since the sum of N4 and N7 gives $|\max_t - x_t|$, which is reflected by the weights of the last logit. Note that there is flexibility in the design of s and m; we provide a few other options below.

```
def distilled_algo(test_x):
    s = [0 for _ in range(10)]
    m = [0 for _ in range(10)]
    max_t = 0
    xt = 0

    for t in range(10):
        # rotate
        for k in range(9):
            s[k] = s[k+1]
```

```
            m[k] = m[k+1]

        # update
        xt = test_x.squeeze()[t]
        m[9] = xt
        s[9] = max_t - abs(max_t - xt)
        # other choices can be
        # m[9] = max_t -  max(xt - max_t, 0)
        # m[9] = max(xt, 0)
        # s[9] = max(max_t - xt, 0)

        # exact shifting version
        if xt > max_t:
            for k in range(9):
                s[k] = m[k]

        # approximate shifting version
        # for k in range(9):
        #     s[k] = m[k] - 2 * max(max_t - xt, 0)
        #                 + 2 * max(max_t - m[k] / 2 + s[k] / 2 - xt, 0)

        max_t = max(xt, max_t)

    return max(enumerate(s), key=lambda x: x[1])[0]
```

Listing 1: 2nd argmax algorithm

Error analysis Here, we provide an error analysis of the exact shifting version of the distilled algorithm. The approximate shifting version can be analyzed similarly. This algorithm always computes the argmax instead of the 2nd argmax when the entire sequence is negative. When exactly one entry of x is positive (say the k-th), then $s_k = -x_k$, which is correct as long as the largest negative number is larger than $-x_k$. This has a probability of 0.9. If it is the last entry, then it is correct as long as the largest negative number is larger than $-x_k$. If it is not the last entry, the update $s_k \leftarrow m_k$ does not affect the result because they are the same, reducing to the same case as above. When there are two positive entries, correctness is guaranteed whether the larger one appears earlier or later. Therefore, this algorithm should have accuracy $1-(2^{-10}+0.1\times 10\times 2^{-10}) \approx 0.998$, which matches the empirical accuracy.

Discrepencies Overall, the constructed net and the trained net share similar qualitative patterns. N2, N4 and N7 of the trained net have their exact counterparts in the constructed net. Both nets have a max/argmax sub-circuit. However, there are some major discrepancies:

- The constructed 21-neuron net has an accuracy 99%. In comparison, the trained 16-neuron RNN has an accuracy 94% and empirically we found it hard to train a 21-neuron RNN with 95%+ accuracy.
- Both the constructed and the trained net have 1 major positive entry in the input weight vector w_{ih}. However, the constructed net has 10 negative entries, designated

to update each of the s_k features. In comparison, the trained net has 4 negative input neurons which seem very entangled and this has been a major obstacle to understanding how $(s_k - m_k)$ is represented.

- In the constructed net, the m_k features are stored without updates but the max circuit of the trained net does not store them statically and or preserve the order. We have similar observations for trained RNNs on the argmax task as well.

- As mentioned above, there are potentially a lot of ways of designing the s and m features. The trained net do not seem to implement the versions described in the code.

5.3 Heuristic Estimator

In our attempt, we were primarily focused on complete mechanistic interpretations, which was hindered by understanding the m (max sub-circuit) and $(s - m)$ feature implementations. We shifted our efforts to analyzing RNNs for argmax tasks but did not uncover an explanation for nonlinearity's benefits. An unexplored direction is pursuing a higher-level mechanistic view while leveraging generic end-to-end techniques like bin propagation.

Prior work on end-to-end heuristic estimators includes extracting a deterministic finite-state automaton (DFA) from the RNN. One approach to state space partitioning is to divide each dimension into several intervals. Another approach, aiming to avoid dimensionality issues, is to fit an unsupervised classifier such as k-means to a large sample set of reachable network states. [3]

[3] See, e.g., "Extracting automata from recurrent neural networks using queries and counterexamples." and references therein.